TRƯỜNG ĐẠI HỌC FPT

TRƯỜNG ĐẠI HỌC FPT

UNI-MATE

Software Design Specification

– Ho Chi Minh City, July 2026 –

RECORD OF CHANGES

Date	A*, M, D	In charge	Change Description
14/07/2026	A	UNI-MATE Team	Initial SDS baseline derived from the current React/Express/MongoDB implementation.

** A - Added; M - Modified; D - Deleted*

Table of Contents

I. Overview

UNI-MATE is a full-stack web application for students aged 18 to under 30 to discover compatible people, form mutual matches, communicate, create small groups, choose cafes, and use partner vouchers. The reviewed baseline consists of a React/Vite client, an Express/TypeScript API, MongoDB persistence through Mongoose, and Socket.IO realtime messaging.

Implementation baseline: This SDS documents the code present in the repository on 14 July 2026. It describes actual behavior, including the current mutual-like chat behavior and the remaining cafe proposal workflow.

1. System Architecture and Code Packages

a. Architectural View

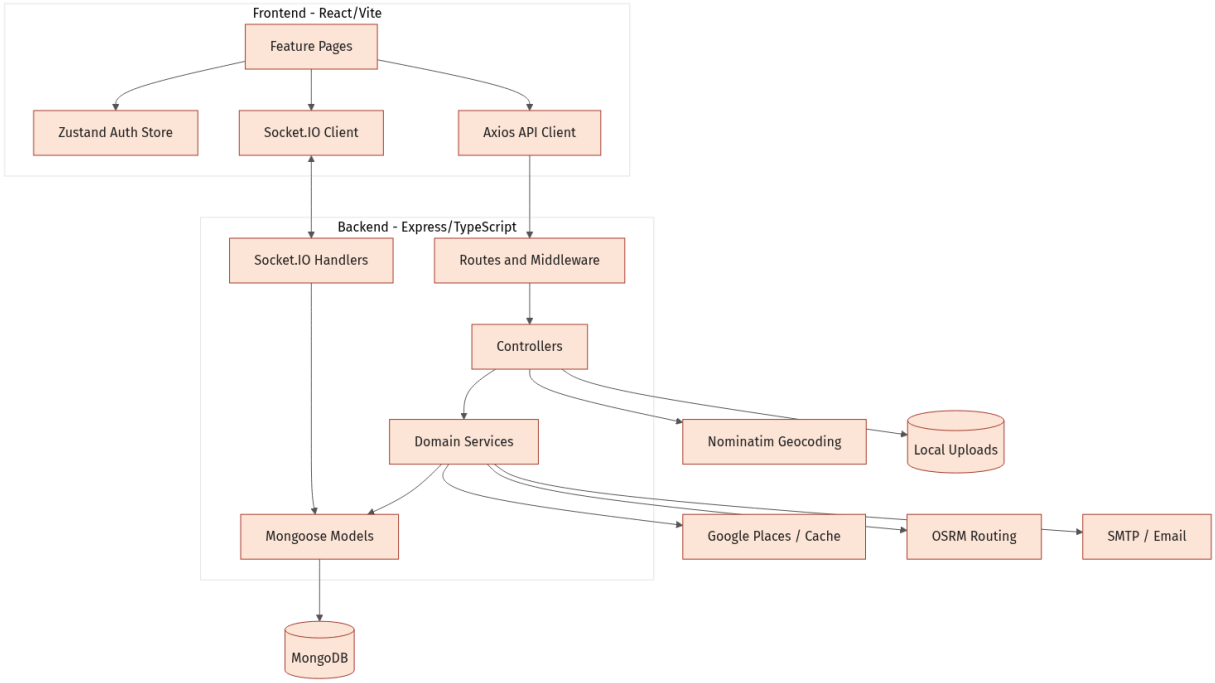

Figure 1. UNI-MATE logical architecture and external integrations

The client uses REST for transactional operations and Socket.IO for realtime chat, typing indicators, read receipts, match updates, and notifications. The backend applies JWT authentication, role checks, Zod validation, rate limiting, and centralized error handling before invoking controllers and services.

b. Package Dependency Diagram

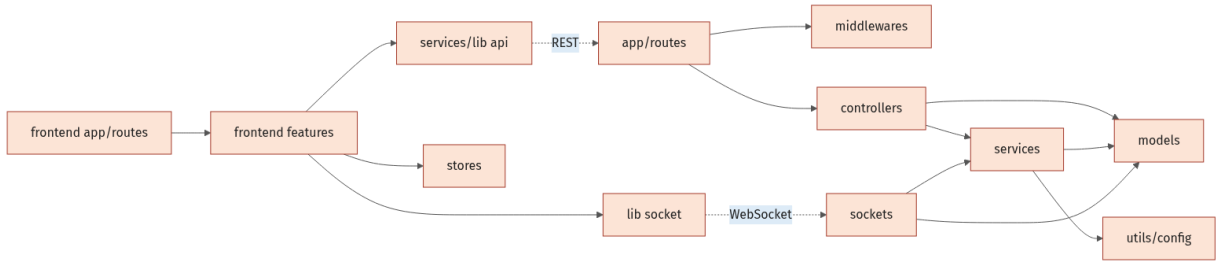

Figure 2. Frontend and backend package dependencies

c. Package Descriptions

No	Package	Description
01	frontend/src/app	Top-level React route composition and application bootstrap.
02	frontend/src/features	Feature screens for authentication, onboarding, discovery, match, chat, group, places, safety, partner, and admin.
03	frontend/src/services + lib	Axios REST client, access-token refresh interceptor, and Socket.IO client.
04	frontend/src/stores	Zustand authentication/session state and pending OTP context.
05	backend/src/routes	HTTP endpoint declarations and composition of authentication/validation middleware.
06	backend/src/controllers	Request orchestration, authorization-sensitive checks, response mapping, and socket emission.
07	backend/src/services	Authentication, compatibility scoring, distance routing, place suggestion, email, and notification logic.
08	backend/src/models	Mongoose schemas, constraints, references, indexes, and lifecycle fields.
09	backend/src/sockets	Authenticated private/group realtime messaging and read/typing events.
10	backend/src/middlewares	JWT authentication, admin authorization, validation, not-found, and error handling.
11	backend/src/validations	Zod request contracts for authentication, users, matching, messages, and safety.
12	backend/src/utils + config	Environment configuration, token helpers, zodiac calculation, seeding, and utility functions.

2. Design Conventions

1. HTTP routes use resource-oriented paths under /api and JSON request/response bodies.
2. Protected routes use Bearer access tokens; refresh tokens are also stored in an HTTP-only cookie.
3. Mongoose ObjectId references model cross-collection relationships; selected aggregates embed arrays and value objects for read efficiency.
4. Dates use server-side Date values and Mongoose timestamps. Geospatial coordinates use GeoJSON [longitude, latitude].
5. Realtime rooms are named with either the ChatRoom id, group:<groupId>, or user:<userId>.
6. Status fields implement lifecycle state rather than physical deletion for matches, rooms, groups, places, reports, and vouchers.

3. Database Design

MongoDB is the system of record. Mongoose schemas enforce required fields, enumerated statuses, selected uniqueness constraints, TTL behavior for OTP records, and 2dsphere indexes for user and cafe locations.

a. Core Matching and Chat Schema

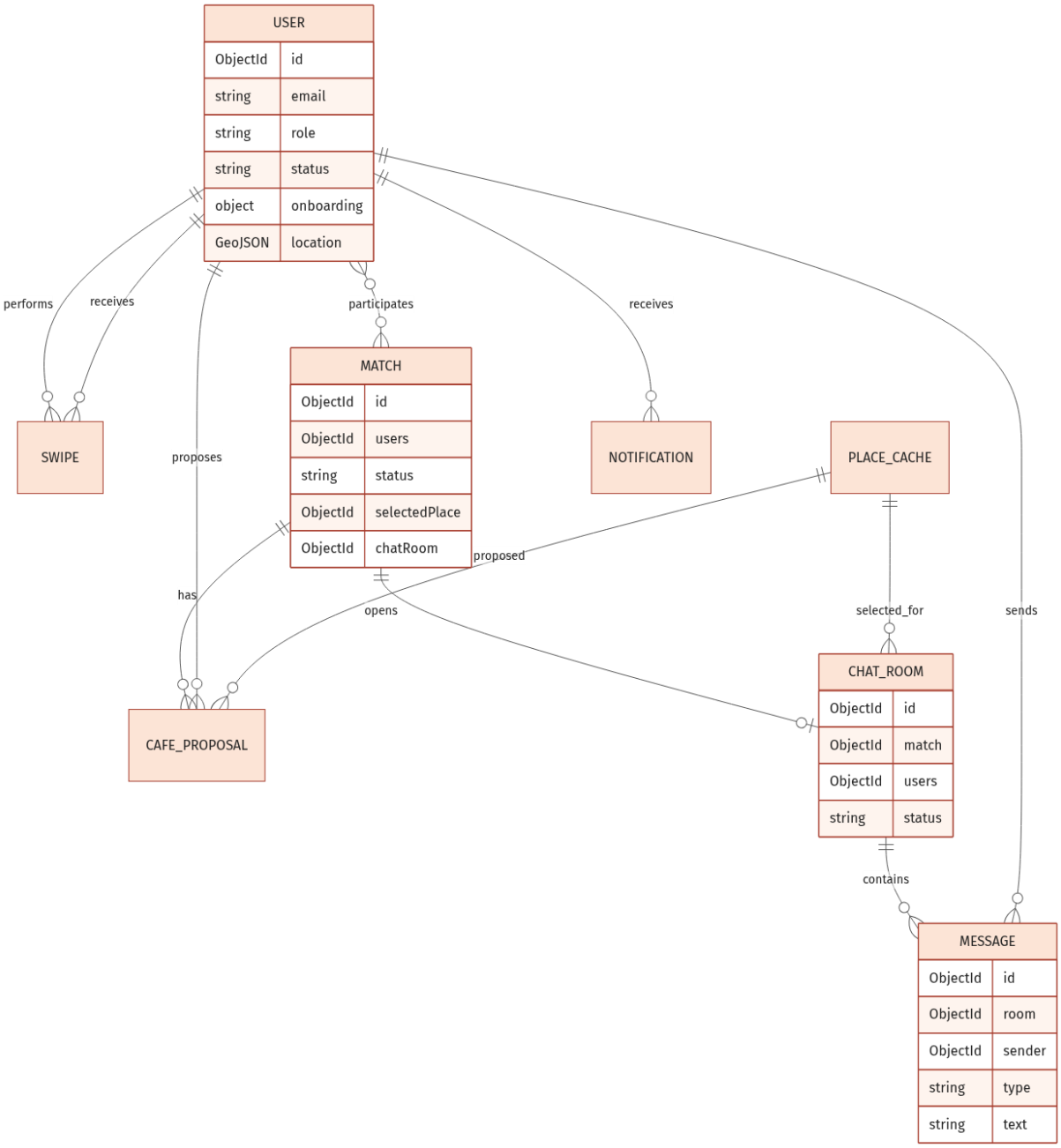

Figure 3. Core user discovery, match, cafe proposal, and private chat collections

b. Extended Social, Partner, and Moderation Schema

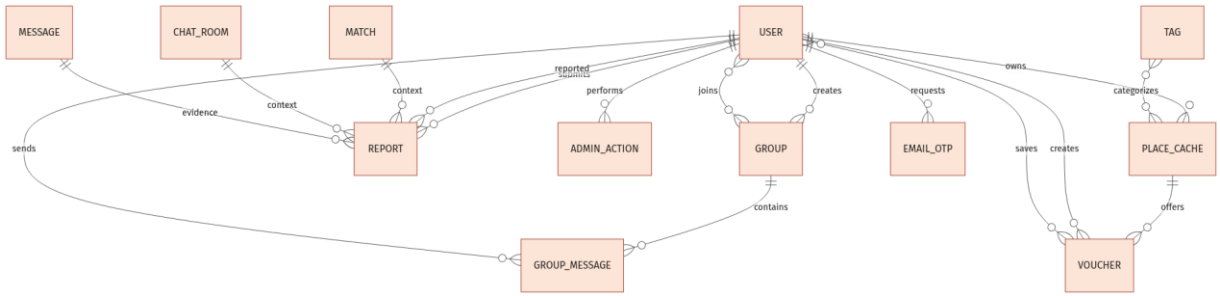

Figure 4. Group, report, partner, voucher, tag, and OTP relationships

c. Collection Descriptions

No	Collection	Purpose
01	User	Identity, role/status, profile, onboarding survey, location, preferences, block list, and token metadata.
02	EmailOtp	Hashed email OTP with expiry, attempt count, resend count, consumed flag, and TTL cleanup.
03	Otp	Legacy/simple hashed OTP record keyed by email with TTL expiry.
04	Swipe	Unique directed like/pass decision from one user to another.
05	Match	Two participants, compatibility score, lifecycle state, selected cafe, confirmations, room, and expiry.
06	CafeProposal	Cafe proposed for a match, proposer, cafe reference, and active/accepted/rejected history.
07	ChatRoom	One-to-one match room, participants, cafe, activity state, hide list, and last-message summary.
08	Message	Private-room text or file message, sender, type, attachment metadata, and readBy list.
09	Notification	Per-user notification with type, title/body, flexible data payload, and read timestamp.
10	Group	Owner-created group, member list, purpose, capacity, and active/dissolved state.
11	GroupMessage	Group text/file message with sender and readBy list.
12	PlaceCache	Cafe/place profile, address, geolocation, operational fields, tags, amenities, and partner ownership.
13	Voucher	Partner-issued cafe voucher, quota, usage count, saved users, expiry, and activation state.
14	Report	Reporter, reported user, optional match/room/message context, evidence, priority, and moderation result.
15	AdminAction	Append-only audit event with admin, action, polymorphic target type/id, and reason.
16	Tag	Admin-managed cafe/profile tag catalogue; places currently persist tag names as strings.

d. Important Indexes and Constraints

Collection	Index / constraint	Rationale
User	email unique; role/status/isActive; location 2dsphere	Authentication, admin filtering, and discovery radius queries.
Swipe	fromUser + toUser unique	One current directed decision per user pair.
Match	users; status	Participant lookup and lifecycle filtering.

Collection	Index / constraint	Rationale
ChatRoom	match unique	At most one room per match.
CafeProposal	match + status	Efficient retrieval/replacement of active proposal.
PlaceCache	googlePlaceId unique sparse; location 2dsphere	Cache identity and nearby cafe search.
Voucher	code + placeId unique	Prevents duplicate code within a cafe.
Report	status + createdAt	Moderation queue ordering.
EmailOtp	email + createdAt; expiresAt TTL	Latest challenge lookup and automatic expiry.

II. Code Designs

This section groups classes and runtime interactions by user-visible function. Boundary objects are React pages, control objects are controllers/services/socket handlers, and entity objects are Mongoose models.

1. Authentication, Account, and Onboarding

This subsystem creates accounts, validates credentials, optionally performs email OTP two-factor authentication, refreshes sessions, resets passwords, and collects the profile/preferences/location required for discovery.

a. Class/Component Diagram

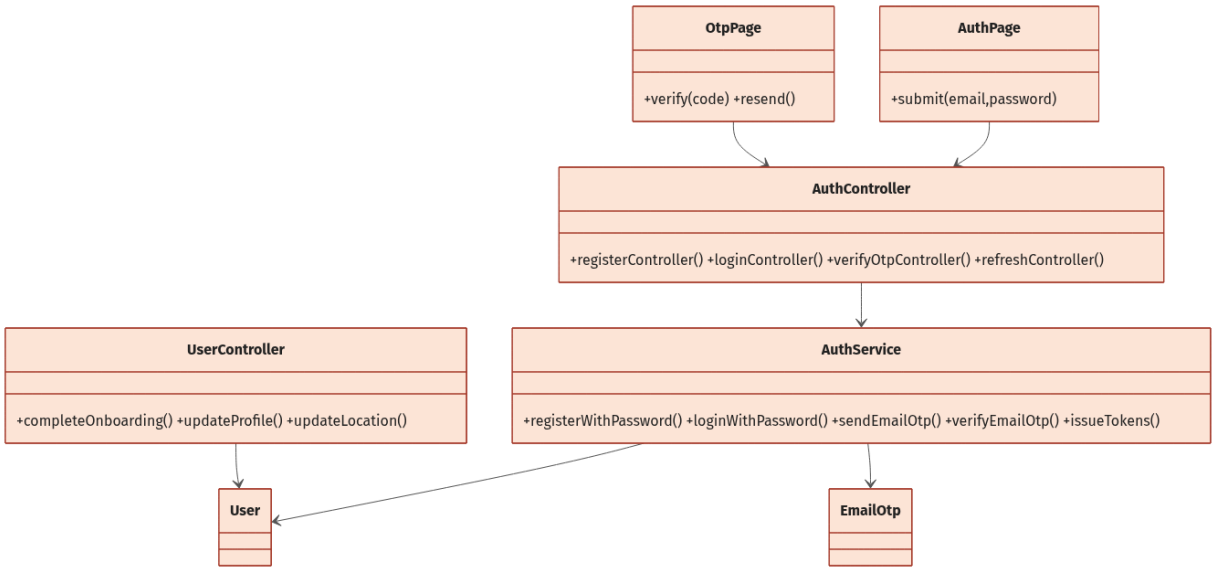

Figure 5. Authentication and profile component relationships

b. Class and Method Specifications

Component	Method	Inputs / outputs and internal processing
AuthController	registerController	email, password -> user/tokens; delegates validation and registration, then sets refresh cookie.
AuthController	loginController	credentials -> tokens or requiresTwoFactor; sets refresh cookie only after full authentication.
AuthService	registerWithPassword	Normalizes email, rejects duplicate, hashes password, creates verified user, and issues tokens.
AuthService	loginWithPassword	Restores expired suspension, checks account state/password, and branches to OTP or token issue.
AuthService	sendEmailOtp	Generates six-digit OTP, stores bcrypt hash and expiry, enforces resend metadata, and sends email.
AuthService	verifyEmailOtp	Validates newest unconsumed OTP, increments failed attempts, consumes success, and issues tokens.
UserController	completeOnboarding	Profile/survey/location -> updated User; enforces age 18 to under 30 and computes zodiac.
UserController	geocodeLocation	city/district -> filtered Vietnam coordinates via Nominatim.
UserController	updateProfile	Whitelisted profile/preferences values -> persisted user.

c. Sequence Diagram(s)

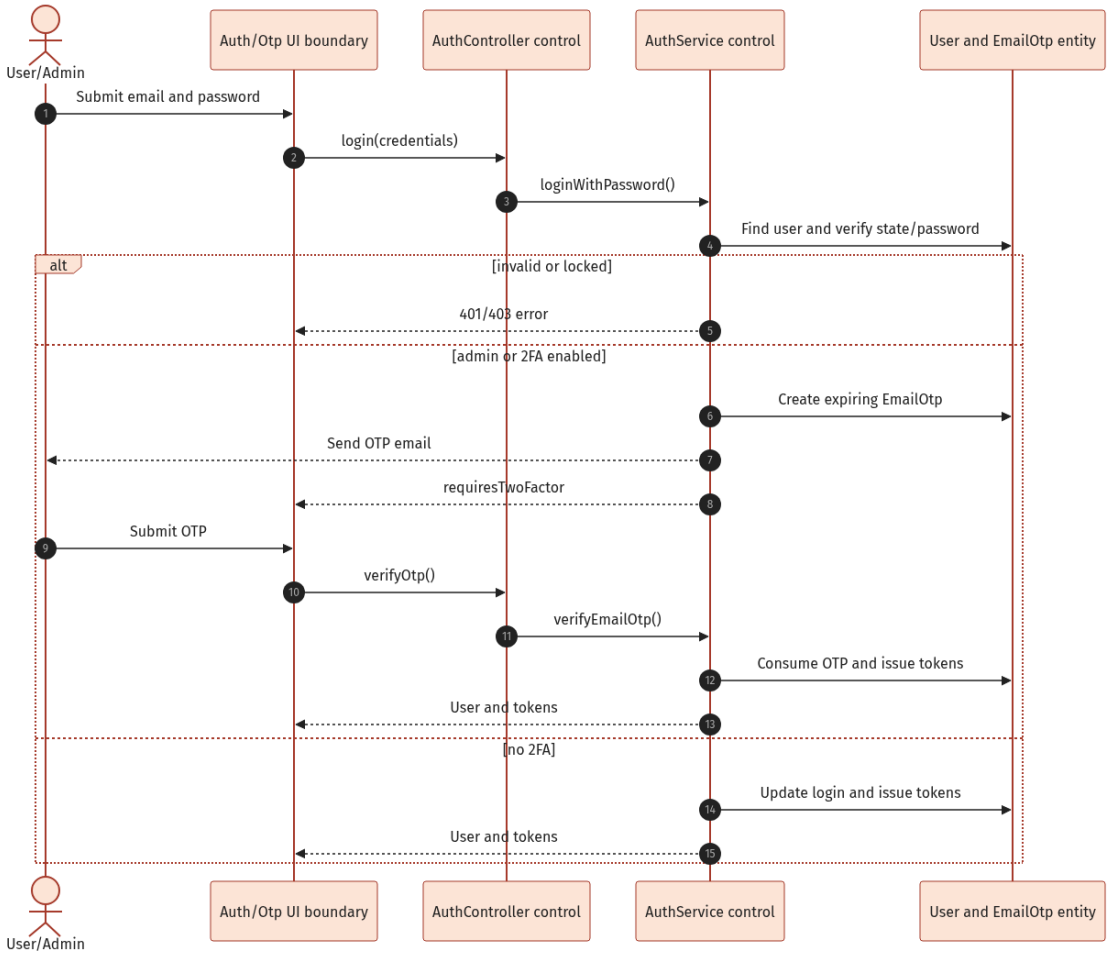

Figure 6. Password login with optional email OTP two-factor authentication

d. Database Queries / Persistence Operations

Representative operations used by the implementation are shown below. Mongoose translates these operations to MongoDB queries.

```
const user = await User.findOne({ email });
await EmailOtp.create({ email, otpHash, expiresAt });
const otp = await EmailOtp.findOne({ email, consumed: false, expiresAt: { $gt: new Date() } })
  .sort({ createdAt: -1 });
await User.findByIdAndUpdate(userId, { onboardingCompleted: true, onboarding, location });
```

2. Discovery and Mutual Matching

Discovery excludes prior swipes, both directions of blocking, inactive/non-user accounts, incomplete onboarding, invalid locations, incompatible age/purpose/gender filters, and users outside the configured radius. Candidates are ranked with route-time distance, cafe style, interests, goals, major preference, vibe, age, recency, and time overlap.

a. Class/Component Diagram

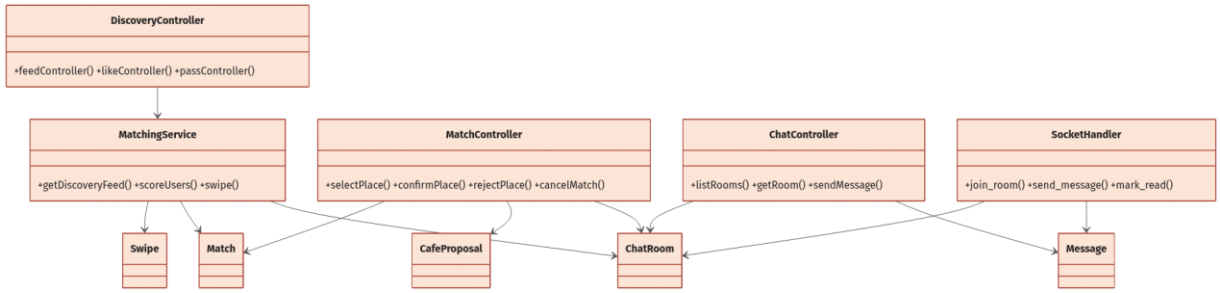

Figure 7. Discovery, match, cafe proposal, and chat control/entity relationships

b. Class and Method Specifications

Component	Method	Inputs / outputs and internal processing
DiscoveryController	feedController	Authenticated user -> ranked candidate list.
DiscoveryController	likeController	targetUserId -> like result; emits incoming-like notification when not mutual.
MatchingService	getDiscoveryFeed	Loads exclusion/filter sets, geospatial candidates, OSRM route metadata, scores, sorts, and limits to 10.
MatchingService	scoreUsers	Two profiles and route metadata -> score, reasons, common tags/styles, distance diagnostics.
MatchingService	swipe	Upserts directed action; on mutual like creates/upserts Match and active ChatRoom.
MatchingService	getIncomingLikes	Returns users who liked the current user and have not received a response swipe.

c. Sequence Diagram(s)

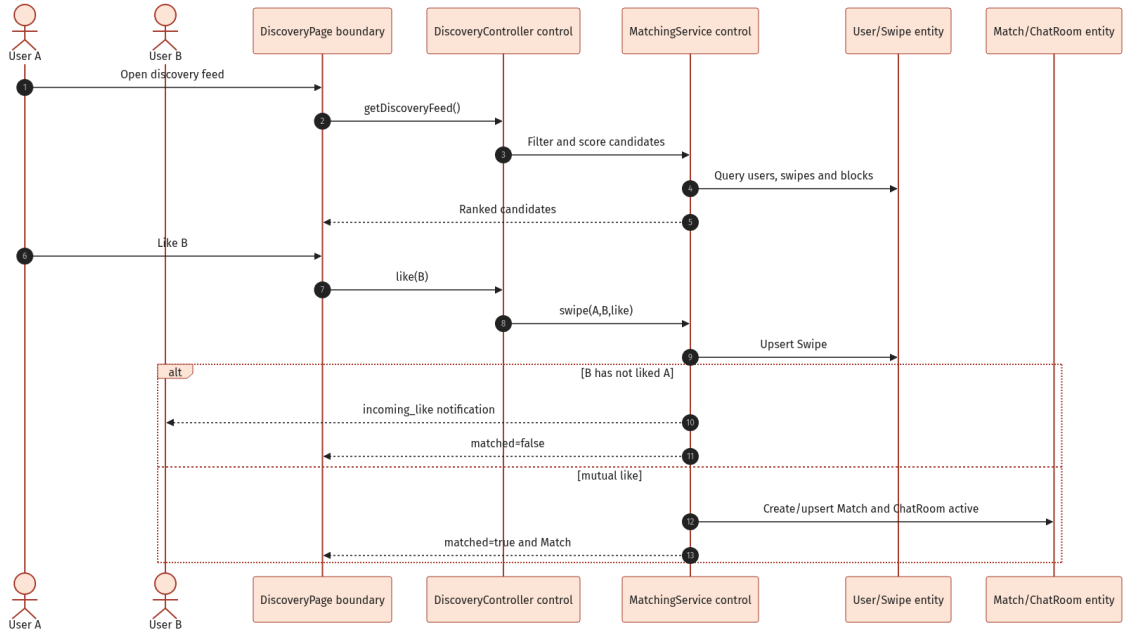

Figure 8. Discovery like/pass decision and actual mutual-match behavior

d. Database Queries / Persistence Operations

Representative operations used by the implementation are shown below. Mongoose translates these operations to MongoDB queries.

```
const swiped = await Swipe.find({ fromUser: userId }).distinct("toUser");
const users = await User.find({ _id: { $nin: excluded }, role: "user", status: "active", location:
geoFilter });
await Swipe.findOneAndUpdate({ fromUser, toUser }, { action }, { upsert: true, new: true });
const reciprocal = await Swipe.findOne({ fromUser: target, toUser: actor, action: "like" });
await Match.create({ users: [actor, target], status: "chat_opened", expiresAt });
```

Actual behavior: Mutual like currently creates an active ChatRoom immediately. This conflicts with the README rule that chat should remain locked until both users confirm the same cafe.

3. Cafe Proposal and Match Lifecycle

A participant can retrieve suggested cafes, replace the active proposal, and notify the other participant. The proposal creator is automatically recorded as the first confirmer. Only the other participant can confirm or reject. Confirmation upserts an active room; rejection clears the selection; cancellation archives an existing room.

a. Class/Component Diagram

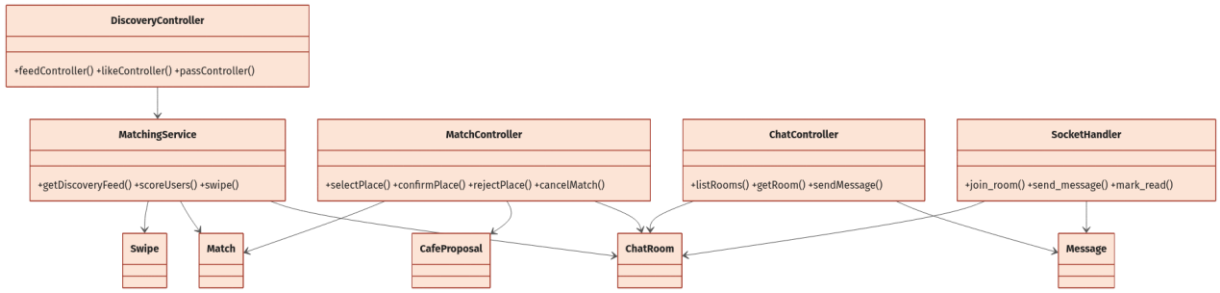

Figure 9. MatchController relationships to Match, CafeProposal, PlaceCache, and ChatRoom

b. Class and Method Specifications

Component	Method	Inputs / outputs and internal processing
MatchController	placeSuggestions	matchId -> suggested active cafes; rejects missing/expired match.
MatchController	selectPlace	matchId/placeId -> proposal; replaces active proposal and sets cafe_proposed.
MatchController	confirmPlace	Adds non-proposer to confirmedBy, accepts proposal, upserts room, and sets chat_opened.
MatchController	rejectPlace	Non-proposer rejects; clears selection/confirmations and returns match to matched.
MatchController	cancelMatch	Sets cancelled, rejects active proposals, archives room, and notifies peer.
PlacesService	suggestCafePlaces	Uses participant locations and cached/seeded places to return cafe candidates.

c. Sequence Diagram(s)

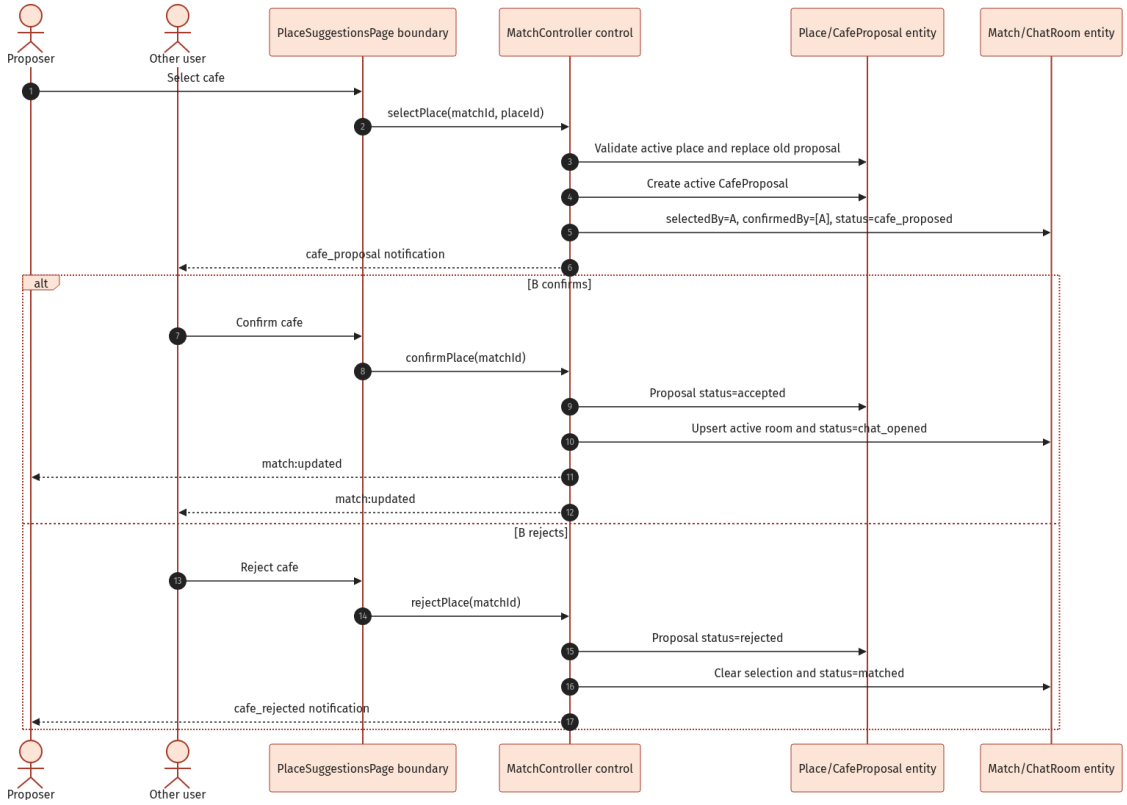

Figure 10. Cafe selection, confirmation, rejection, and realtime match update

d. Database Queries / Persistence Operations

Representative operations used by the implementation are shown below. Mongoose translates these operations to MongoDB queries.

```
await CafeProposal.updateMany({ match: match._id, status: "active" }, { status: "replaced" });
await CafeProposal.create({ match: match._id, proposedBy: userId, cafe: placeId, status: "active" });
await Match.findOneAndUpdate({ _id: matchId }, { $addToSet: { confirmedBy: userId } }, { new: true
});
await ChatRoom.findOneAndUpdate({ match: matchId }, roomData, { upsert: true, new: true });
```

4. Private Chat and Notifications

Private chat supports room listing/hiding, chat history, text and file messages, typing indicators, delivery broadcasts, read receipts, and per-recipient notifications. Both HTTP and Socket.IO message submission paths enforce active room and chat_opened match state.

a. Class/Component Diagram

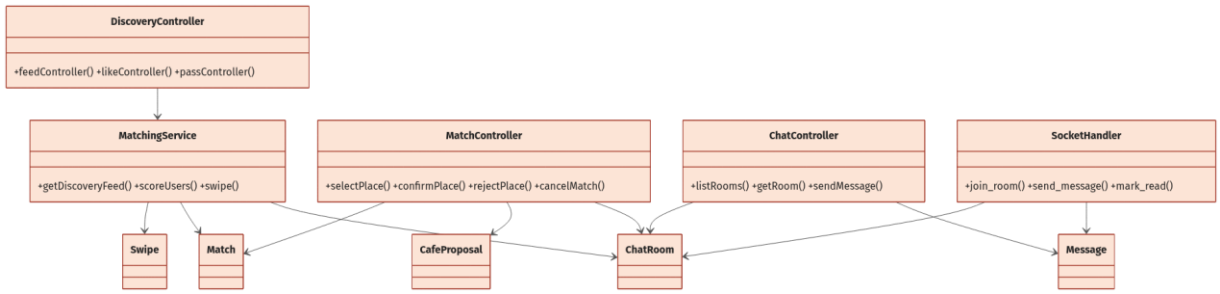

Figure 11. ChatController and SocketHandler relationships to room and message entities

b. Class and Method Specifications

Component	Method	Inputs / outputs and internal processing
ChatController	listRooms	Returns rooms containing user and not hidden by that user, ordered by lastMessageAt.
ChatController	getRoom	Verifies membership and states, calculates bilateral block state, and returns up to 100 messages.
ChatController	sendMessage	HTTP fallback: validates message, persists it, updates room, broadcasts, and notifies peer.
SocketHandler	join_room	Joins room only when the socket user is a participant and room is active.
SocketHandler	send_message	Realtime validation/persistence/broadcast plus notification creation.
SocketHandler	mark_read	Adds reader id to readBy for all room messages and emits message_read.
NotificationService	createAndEmitNotification	Persists Notification then emits notification:new to user:<id>.

c. Sequence Diagram(s)

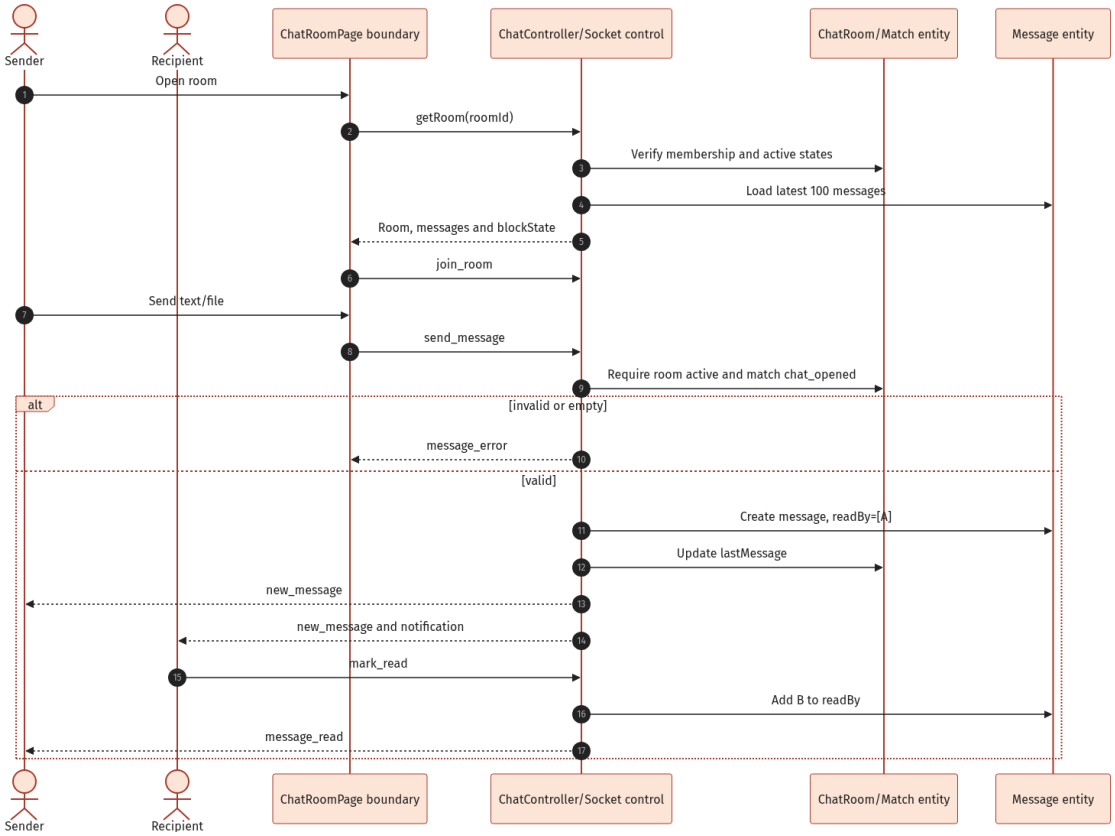

Figure 12. Realtime private message, notification, and read-receipt flow

d. Database Queries / Persistence Operations

Representative operations used by the implementation are shown below. Mongoose translates these operations to MongoDB queries.

```
const room = await ChatRoom.findOne({ _id: roomId, users: userId });
const match = await Match.findById(room.match);
const message = await Message.create({ room: room._id, sender: userId, text, type, fileUrl, readBy:
[userId] });
await Message.updateMany({ room: roomId }, { $addToSet: { readBy: userId } });
```

5. Groups and Group Chat

Users can create purpose-oriented groups, add members by email, remove members or leave, dissolve a group, and exchange realtime or HTTP-fallback group messages. Owner, capacity, membership, and active-state rules are enforced in controllers/socket handlers.

a. Class/Component Diagram

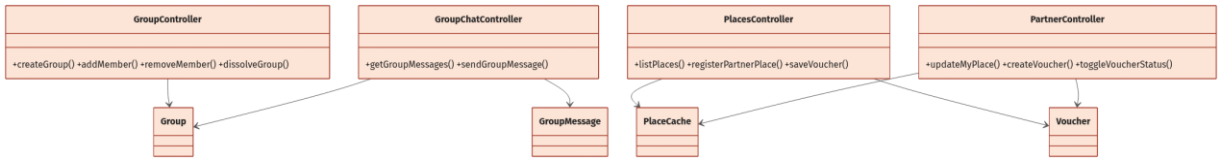

Figure 13. Group management and group-chat components

b. Class and Method Specifications

Component	Method	Inputs / outputs and internal processing
GroupController	createGroup	Creates group with requester as creator and first member; default capacity is six.
GroupController	getMyGroups	Lists active groups containing the requester and populates creator/members.
GroupController	addMember	Owner-only; validates capacity and active target user found by email.
GroupController	removeMember	Owner may remove others; member may leave; owner cannot leave without dissolving.
GroupController	dissolveGroup	Owner-only transition to dissolved.
GroupChatController	getGroupMessages	Verifies member and returns group history.
SocketHandler	send_group_message	Validates active membership, persists GroupMessage, broadcasts, and notifies peers.

c. Sequence Diagram(s)

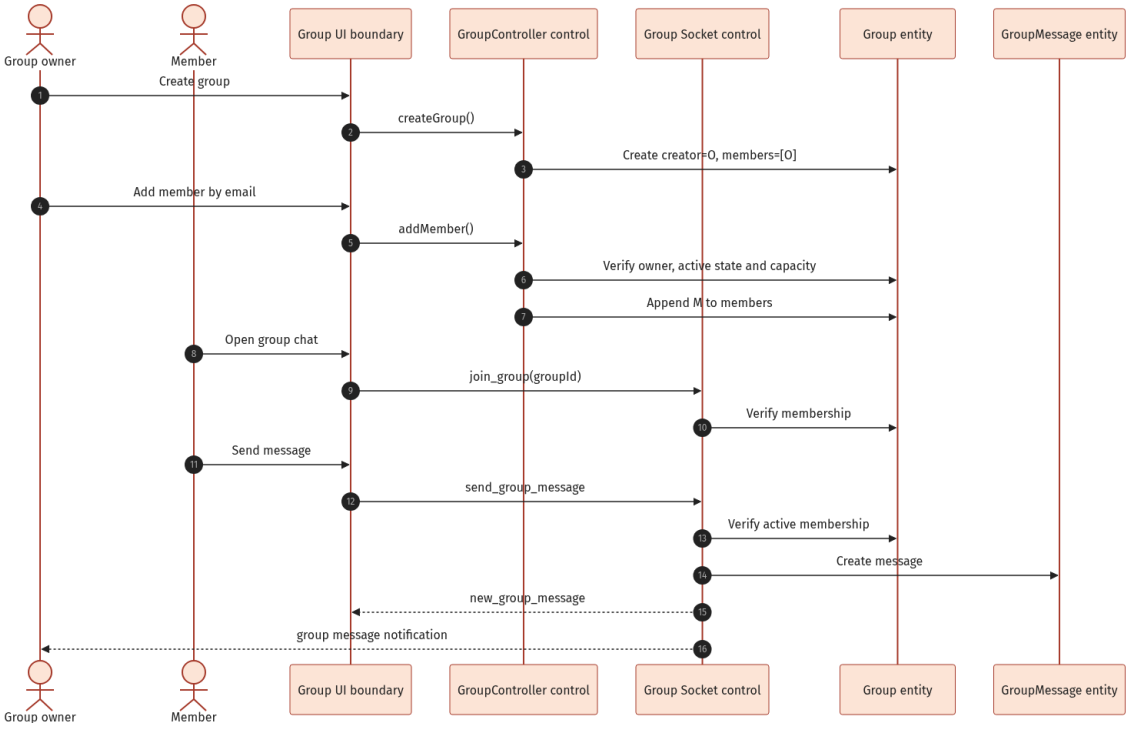

Figure 14. Group creation, member addition, and realtime group messaging

d. Database Queries / Persistence Operations

Representative operations used by the implementation are shown below. Mongoose translates these operations to MongoDB queries.

```
const group = await Group.create({ name, creator: userId, members: [userId], purpose, maxMembers });
const target = await User.findOne({ email: normalizedEmail, status: "active" });
group.members.push(target._id); await group.save();
const messages = await GroupMessage.find({ group: groupId }).sort({ createdAt: 1 });
```

6. Places, Partner Registration, and Vouchers

Authenticated users browse active cafes and available vouchers. A prospective owner submits one pending partner-place application; admin approval activates the place and upgrades the owner role to partner. Partners manage owned place data and voucher lifecycle. Users save valid vouchers atomically against quota.

a. Class/Component Diagram

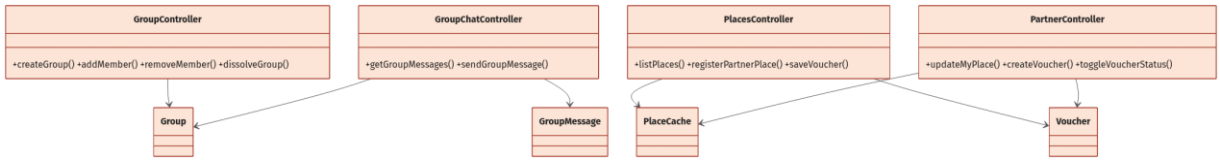

Figure 15. Places, partner ownership, and voucher component relationships

b. Class and Method Specifications

Component	Method	Inputs / outputs and internal processing
PlacesController	listPlaces/getPlace	Returns only active place records.
PlacesController	registerPartnerPlace	Validates cafe form/hours and creates one pending partner place per applicant.
PlacesController	getPlaceVouchers	Returns active, unexpired, non-exhausted vouchers and savedByMe flag.
PlacesController	saveVoucher	Idempotent save; atomically adds user and increments usage only while valid.
PartnerController	getMyPlaces/updateMyPlace	Lists owned places and updates whitelisted fields after ownership check.
PartnerController	createVoucher	Requires partner ownership, active cafe, required values, and unique code within cafe.
PartnerController	toggleVoucherStatus/deleteVoucher	Owner-scoped voucher activation toggle or deletion.
AdminController	hidePlace	Transitions place status, upgrades partner role on approval, notifies owner, and audits.

c. Sequence Diagram(s)

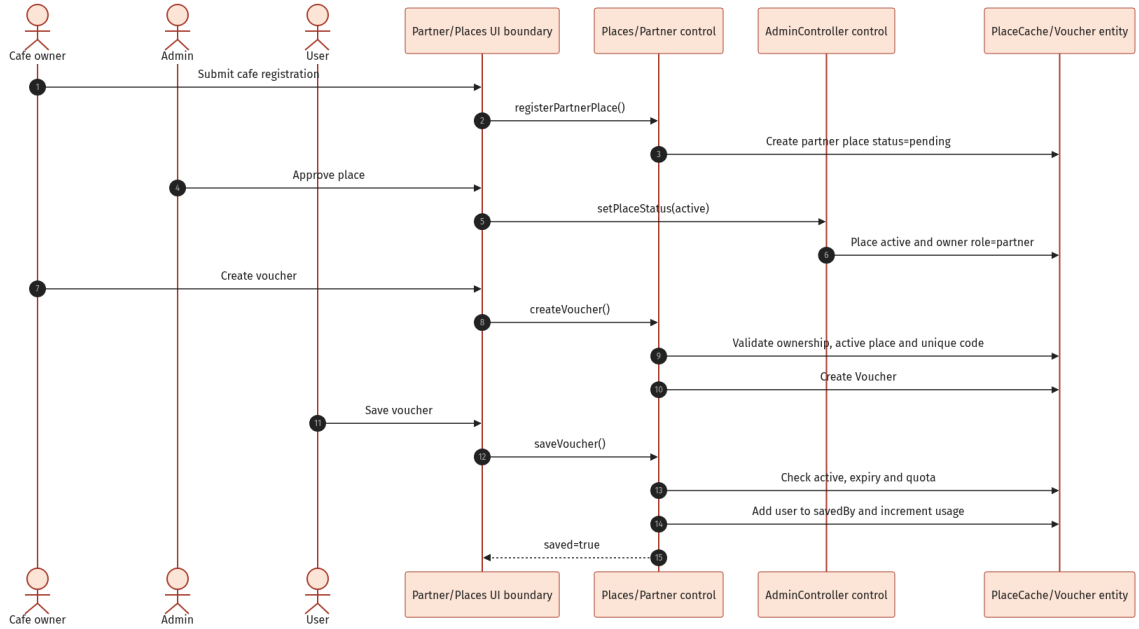

Figure 16. Partner place approval, voucher creation, and user save flow

d. Database Queries / Persistence Operations

Representative operations used by the implementation are shown below. Mongoose translates these operations to MongoDB queries.

```
const place = await PlaceCache.create({ ...data, partnerId: userId, isPartnerPlace: true, status:
"pending" });
const existing = await Voucher.findOne({ placeId, code: code.toUpperCase() });
const voucher = await Voucher.findOneAndUpdate(validVoucherFilter, { $addToSet: { savedBy: userId },
$inc: { currentUsageCount: 1 } }, { new: true });
```

7. Safety, Blocking, Reporting, and Moderation

Safety operations validate report context, prioritize repeated independent reports, block communication across related matches/rooms, and restore communication only when neither party continues blocking. Admin moderation supports dismiss, warn, suspend, and ban with warning-count escalation and audit logging.

a. Class/Component Diagram

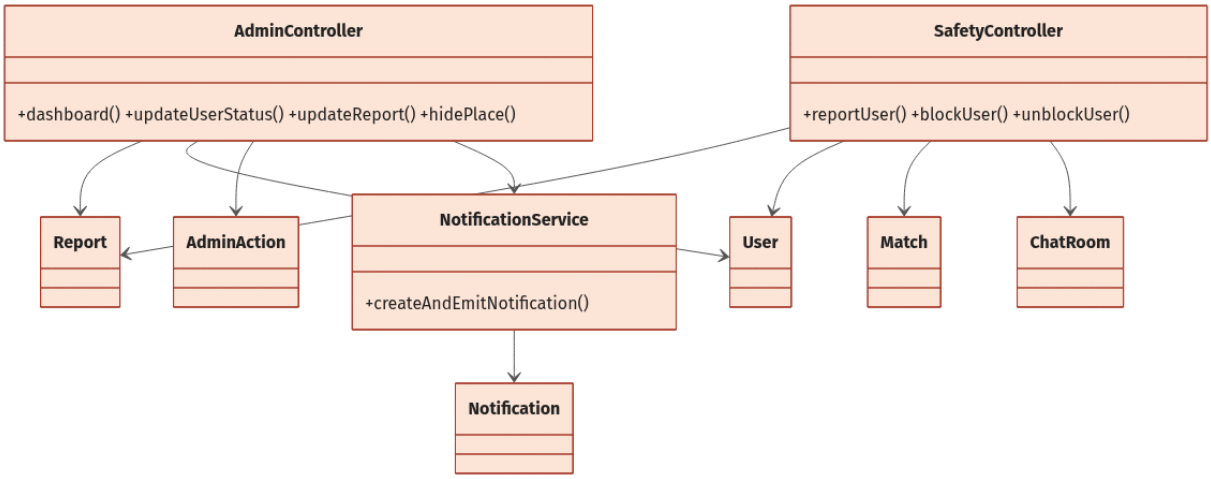

Figure 17. Safety and moderation control/entity relationships

b. Class and Method Specifications

Component	Method	Inputs / outputs and internal processing
SafetyController	reportUser	Creates report with optional match/room/message/evidence and marks priority after three distinct reporters.
SafetyController	blockUser	Adds target to blockedUsers and sets all pair Matches/ChatRooms to blocked.
SafetyController	unblockUser	Removes block; restores chat_opened/active only if target does not still block requester.
AdminController	adminReportDetail	Loads report context and up to 200 room messages for investigation.
AdminController	updateReport	Applies one terminal action, updates account/report, emits notice, disconnects locked user.
AdminController	audit	Creates AdminAction with admin/action/targetType/targetId/reason.

c. Sequence Diagram(s)

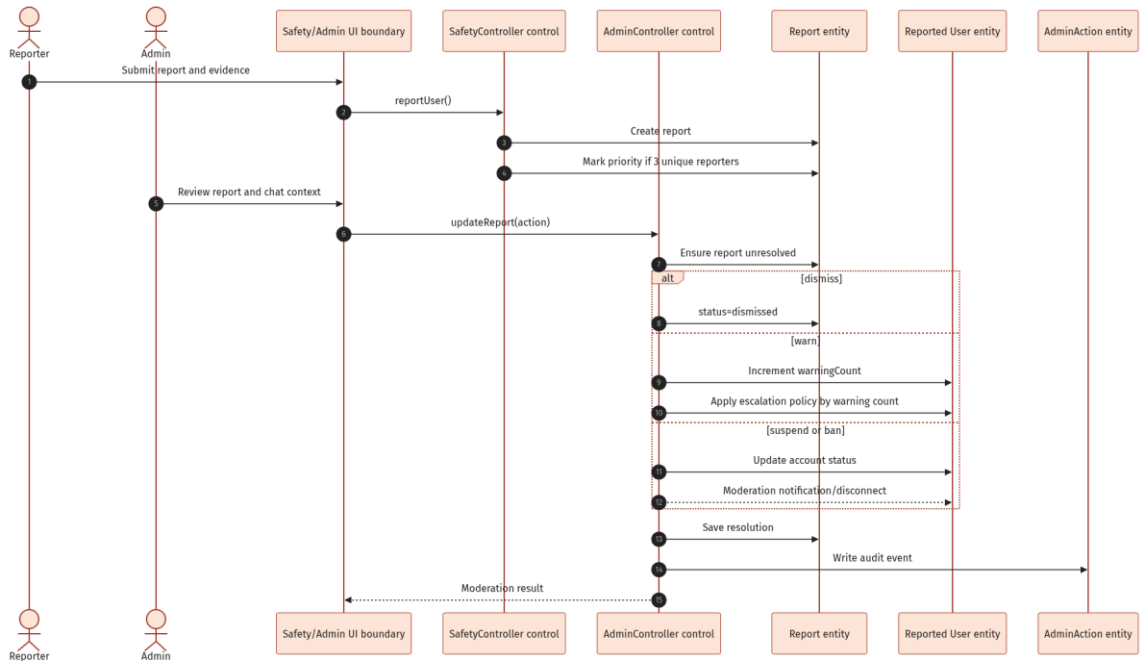

Figure 18. Report submission and admin moderation decision

d. Database Queries / Persistence Operations

Representative operations used by the implementation are shown below. Mongoose translates these operations to MongoDB queries.

```
const report = await Report.create({ reporter, reportedUser, match, room, message, reason,
evidenceUrls });
const reporters = await Report.distinct("reporter", { reportedUser });
await User.findByIdAndUpdate(actor, { $addToSet: { blockedUsers: target } });
await Match.updateMany({ users: { $all: [actor, target] } }, { status: "blocked" });
await AdminAction.create({ admin, action, targetType, targetId, reason });
```

8. Administration and Operational Oversight

Admin pages provide dashboard counts, user search/detail/update/status control, report queue/detail/resolution, match search, place review/update/delete/status control, and audit log listing. All admin routes require authentication and the admin role.

a. Class/Component Diagram

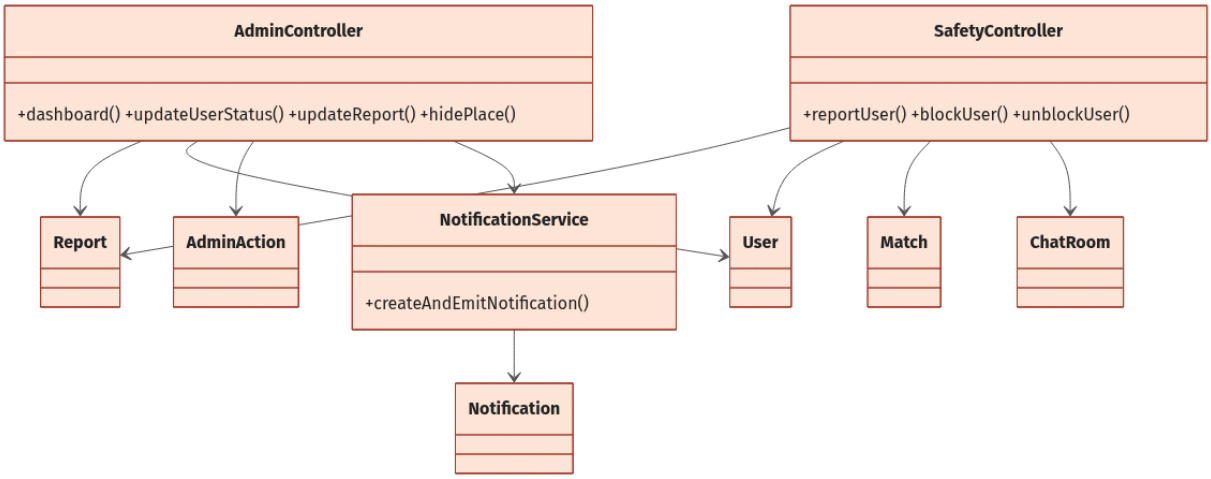

Figure 19. Administration controller dependencies and audited targets

b. Class and Method Specifications

Component	Method	Inputs / outputs and internal processing
AdminController	dashboard	Parallel counts for users, new users, matches, confirmed/opened matches, new reports, and active places.
AdminController	adminUsers/adminUserDetail	Filters users and loads selected user's match/report history.
AdminController	createAdminUser/updateAdminUser	Admin-managed account creation and full editable profile/role/status update.
AdminController	updateUserStatus	Transitions active/suspended/banned and records audit reason.
AdminController	adminMatches	Filters by status or searches participant, cafe, object id, and status text.
AdminController	adminPlaces/upsertPlace/hidePlace/deletePlace	Review and lifecycle management of cafe records.
AdminController	adminActions	Returns latest 100 populated audit events.

c. Sequence Diagram(s)

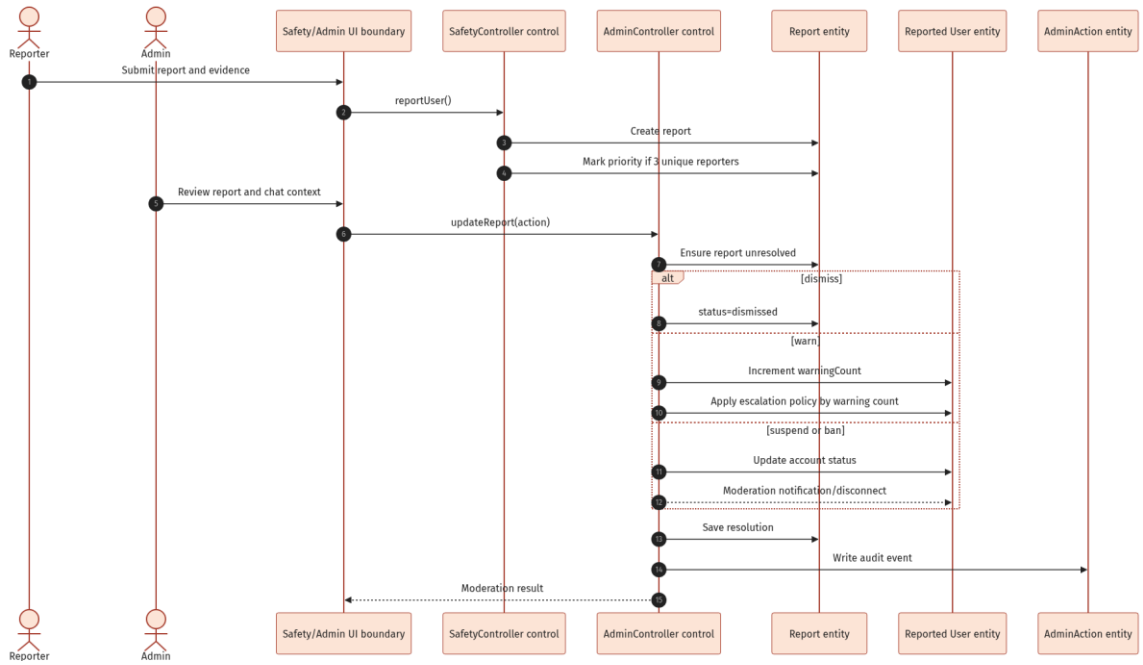

Figure 20. Representative admin review and audited moderation flow

d. Database Queries / Persistence Operations

Representative operations used by the implementation are shown below. Mongoose translates these operations to MongoDB queries.

```
const [users, matches, reports, places] = await Promise.all([User.countDocuments(),
Match.countDocuments(), Report.countDocuments({ status: "new" }), PlaceCache.countDocuments({ status:
"active" })]);
const result = await User.findByIdAndUpdate(userId, statusUpdate, { new: true });
const actions = await AdminAction.find().populate("admin", "email displayName").sort({ createdAt: -1
}).limit(100);
```

III. Cross-Cutting Design

1. Security and Authorization

Concern	Current design
Authentication	Short-lived JWT access token; refresh token hash persisted and HTTP-only refresh cookie supported.
Authorization	requireAuth on protected routers; requireAdmin and requirePartner on privileged areas; ownership/member checks inside controllers.
Password/OTP	bcrypt password and OTP hashes; expiring/consumable email OTP; attempt limit; admin always requires 2FA.
Input validation	Zod request validation; Mongoose enum/range constraints; upload type/size checks.
HTTP hardening	Helmet, CORS with configured client origin, 2 MB JSON limit, cookie parser, and 500 requests per 15-minute rate limit.
Account enforcement	Suspended/banned/inactive users cannot log in; moderated sockets are disconnected.
Privacy	Password and refresh hashes excluded from normal/admin projections; report context restricted to authorized admin routes.

2. Realtime Design

Event	Direction	Responsibility
join_room	Client -> Server	Verify private-room membership and active state, then join room id.
send_message	Client -> Server	Persist and broadcast private message; notify other participant.
new_message	Server -> Room	Deliver populated private message.
typing_start/stop	Bidirectional	Broadcast transient private typing state.
mark_read/message_read	Bidirectional	Persist readBy and inform peer.
join_group	Client -> Server	Verify active group membership and join group:<id>.
send_group_message	Client -> Server	Persist/broadcast group message and notify other members.
new_group_message	Server -> Group	Deliver populated group message.
group_typing_start/stop	Bidirectional	Broadcast group typing state.
mark_group_read	Client -> Server	Persist group readBy and emit receipt.
notification:new	Server -> User	Push persisted notification to user:<id>.
match:updated	Server -> Users	Push cafe/match/chat lifecycle change.

3. State Models

Aggregate	Transitions	Notes
Match	matched -> cafe_proposed -> chat_opened; any -> expired/blocked/cancelled	Mutual like currently creates chat_opened directly.
CafeProposal	active -> accepted/replaced/expired/rejected	Only one active proposal is intended per match.
ChatRoom	active <-> blocked; active -> archived	Cancel archives; bilateral unblock may reactivate.
Group	active -> dissolved	Dissolved groups reject joins/messages.
PlaceCache	pending -> active/hidden/rejected	Active partner place upgrades owner to partner.
Report	new -> reviewing -> resolved_valid/resolved_invalid/dismissed	Controller currently directly uses dismissed/resolved_valid for final actions.
User	active -> suspended/banned; suspended -> active after expiry	Warning policy can escalate automatically.

4. External Services and Failure Handling

Dependency	Use	Fallback / control
OSRM	Travel-time matrix for compatibility scoring.	Failure is logged; distance score falls back to zero.
Google Places / cache	Cafe suggestions and metadata.	Cached or seeded fallback places keep the flow testable.
Nominatim	Manual district/city geocoding.	Non-2xx returns 502; results are filtered to Vietnam/city.
SMTP/email	OTP delivery.	Development may print OTP when configured; service errors propagate through centralized handler.
Local uploads	Chat files and report evidence.	20 MB chat limit; 5 MB image-only report evidence.

Appendix A. REST API Inventory

All routes below are prefixed with /api. Except for register/login/OTP/refresh, routes require an authenticated access token. Admin and partner groups apply additional role checks.

Area	Method	Path	Purpose
Auth	POST	/auth/register	Create account and issue tokens.
Auth	POST	/auth/login	Password login; may require 2FA.
Auth	POST	/auth/send-otp	Send email OTP.
Auth	POST	/auth/verify-otp	Verify OTP and issue tokens.
Auth	POST	/auth/forgot-password/send-otp	Start reset.
Auth	POST	/auth/forgot-password/verify-otp	Return reset token.
Auth	POST	/auth/forgot-password/reset	Set new password.
Auth	POST	/auth/refresh	Rotate/refresh session.
Auth	GET/POST	/auth/me; /auth/logout	Current user; terminate session.
Users	GET/PATCH	/users/profile	Read/update profile.
Users	PUT/PATCH	/users/photos; /users/password	Photos and password.
Users	DELETE	/users/profile	Delete/deactivate account.
Users	POST	/users/onboarding	Complete profile/survey/location.
Users	POST/PATCH	/users/location/geocode; /users/location	Geocode/update location.
Users	POST	/users/avatar; /users/photo	Upload images.
Discovery	GET	/discovery; /discovery/incoming-likes	Feed and pending incoming likes.
Discovery	POST	/discovery/like; /discovery/pass	Persist swipe and possibly match.
Matches	GET	/matches; /matches/:id	List/detail.
Matches	GET	/matches/:id/place-suggestions	Suggested cafes.
Matches	POST	/matches/:id/select-place	Create/replace proposal.
Matches	POST	/matches/:id/confirm-place	Accept proposal.
Matches	POST	/matches/:id/reject-place; /cancel	Reject cafe or cancel match.
Chat	GET	/chat; /chat/:roomId	Room list/detail/history.
Chat	POST/DELETE	/chat/:roomId/messages; /chat/:roomId	Send via HTTP; hide conversation.
Notifications	GET/PATCH	/notifications; /read-all; /:id/read	List and mark read.
Groups	GET/POST	/groups	List/create groups.
Groups	GET	/groups/:id; /groups/:id/messages	Group detail/history.
Groups	POST/DELETE	/groups/:id/members; /members/:userId	Add/remove/leave.
Groups	POST	/groups/:id/dissolve; /groups/:id/messages	Dissolve/send fallback.
Places	GET	/places; /places/:id; /places/:id/vouchers	Browse active places/vouchers.

Area	Method	Path	Purpose
Places	POST	/places/partner-register	Submit partner cafe.
Places	GET	/places/my-partner-registration; /saved-vouchers	Application/saved vouchers.
Places	POST/PATCH	/places/:placeId/vouchers/:voucherId/save	Save voucher.
Partner	GET/PATCH	/partner/places; /places/:placeId	Owned cafes.
Partner	GET/POST	/partner/places/:placeId/vouchers	List/create voucher.
Partner	PATCH/DELETE	/partner/vouchers/:id/toggle; /:id	Toggle/delete voucher.
Safety	POST	/safety/report; /block; /unblock	Report and bilateral safety controls.
Upload	POST	/upload/chat-file; /report-evidence	File/evidence uploads.
Admin	GET	/admin/dashboard; /analytics; /settings	Metrics and settings view.
Admin	GET/POST/PUT/PATCH	/admin/users[/:id][/status]	User administration.
Admin	GET/PATCH	/admin/reports[/:id]	Report review/resolution.
Admin	GET	/admin/matches	Match oversight.
Admin	GET/PUT/PATCH/DELETE	/admin/places[/:id][/status]	Place review/maintenance.
Admin	GET	/admin/actions	Audit log.

Appendix B. Configuration and Deployment

1. Runtime Stack

7. Frontend: React, TypeScript, Vite, React Router, Zustand, Axios, Tailwind CSS, Socket.IO Client.
8. Backend: Node.js, Express, TypeScript, Mongoose, JWT, bcryptjs, Zod, Socket.IO, Multer, Nodemailer.
9. Database: MongoDB with geospatial and TTL indexes.
10. Development entry points: frontend http://localhost:5173 and backend http://localhost:5000.

2. Required Configuration Categories

Category	Configuration responsibility
Database	MongoDB connection URI.
Authentication	JWT access/refresh secrets and token lifetimes; cookie secure flag.
Client	Allowed client origin / frontend URL.
Email	SMTP host, port, credentials, sender and development OTP logging option.
Places	Google Maps/Places API key or cached/seeded fallback.
Distance	OSRM endpoint and scoring thresholds/weights/candidate limit.
Uploads	Writable uploads directory and public /uploads route.

3. Build and Run

11. Provide backend and frontend environment files.
12. Install backend dependencies and start MongoDB or configure MongoDB Atlas.
13. Run the backend development process.
14. Install frontend dependencies and run the Vite client.
15. Verify /health, authentication, protected REST routes, and Socket.IO connectivity.

Appendix C. Known Implementation Discrepancies and Risks

Severity	Issue	Design impact
High	Cafe gate bypass	Mutual like creates Match(status=chat_opened) and active ChatRoom immediately, contrary to the README and admin setting cafeGateRequired=true.
High	State regression on cafe selection	Selecting a cafe after mutual like changes chat_opened back to cafe_proposed, temporarily locking an existing chat.
Medium	Registration verification	Registration sets emailVerified=true without OTP; OTP is only used for 2FA/login and password reset.
Medium	Match participant cardinality	Match.users is an unrestricted array; schema does not enforce exactly two users.

Severity	Issue	Design impact
Medium	Tag consistency	Tag has its own catalogue collection, but PlaceCache stores tag names as strings and does not reference Tag ids.
Medium	Unblock restoration	Unblock restores all blocked pair matches to chat_opened, even if their prior state was not an open chat.
Low	Dual OTP models	Both Otp and EmailOtp exist; active auth service primarily uses EmailOtp.
Low	Local file storage	Uploads are stored on the application host and require persistent/shared storage for scaled deployment.

Recommended decision: Either enforce cafe confirmation before creating ChatRoom, or formally remove the cafe-gated requirement and stop moving chat_opened matches back to cafe_proposed. The current hybrid state should not be treated as a stable business rule.